(continued)

When the solution was rolled out in 2010, some 18 months late, it was a disaster.[8] Tens of thousands of public servants were underpaid, overpaid, or not paid at all. Since the go-live date, over 1,000 payroll staff have been required to carry out some 200,000 manual processes each fortnight to ensure that staff salaries are paid.

In 2012, an independent review found that the project had cost the state over $416 million since going into production and would cost an additional $837 million to fix. This colossal sum included $220 million just to fix the immediate software issues that were preventing the system from delivering its core capability of paying Queensland Health staff what they were owed each month.

Building the right software is made even trickier by one commonly overlooked fact: early on in a project, you usually don't know what the right features are.

1.2.3 The knowledge constraint—dealing with uncertainty

One fact of life in software development is that there will be things you don't know. Changing requirements are a normal part of every software project. Knowledge and understanding about the problem at hand and about how best to solve it increases progressively throughout the project.

In software development, each project is different. There are always new business requirements to cater to, new technological problems to solve, and new opportunities to seize. As a project progresses, market conditions, business strategies, technological constraints, or simply your understanding of the requirements will evolve, and you'll need to change your tack and adjust your course. Each project is a journey of discovery where the real constraint isn't time, the budget, or even programmer hours, but your lack of knowledge about what you need to build and how you should build it. When reality doesn't go according to plan, you need to adapt to reality, rather than trying to force reality to fit into your plan. "When the terrain disagrees with the map, trust the terrain" (Swiss Army proverb).

Users and stakeholders will usually know what high-level goals they want to achieve and can be coaxed into revealing these goals if you take the time to ask. They'll be able to tell you that they need an online ticketing system or a payroll solution that caters to 85,000 different employees. And you can get a feel for the scope of the application you might need to build early on in the project.

But the details are another matter entirely. Although users are quick to ask for specific technical solutions to their problems, they're not usually the best-placed to know what solution would serve them best, or even, for that matter, what solutions exist. Your team's collective understanding of the best way to deliver these capabilities, as

[8] See KPMG, "Review of the Queensland Health Payroll System" (2012), http://delimiter.com.au/wp-content/uploads/2012/06/KPMG_audit.pdf.